

# ASSIGNMENT-10.2

**2303A51042**

**BATCH – 29**

Task Description -1(Error Detection and Correction)

Task:

Use AI to analyze a Python script and correct all syntax and logical errors.

Sample Input Code:

```
def calculate_total(nums)
sum = 0
for n in nums
sum += n
return total
```

Expected Output-1:

Corrected and executable Python code with brief explanations of the identified syntax and logic errors.

**CODE:**

```
10.2(T1).py X
10.2(T1).py > ...
1 def calculate_total(nums):
2     total = 0
3     for n in nums:
4         total += n
5     return total
6
7
8 # Example test
9 print(calculate_total([1, 2, 3, 4]))
```

## OUTPUT:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python
zohaib@MacBook-2 AI ASSISTANT LAB.py % /usr/bin/python3 "/Users/zohaib/Documents/AI ASSISTANT/AI ASSISTANT LAB.py/10.2(T1).py"
10
zohaib@MacBook-2 AI ASSISTANT LAB.py %
```

## OBSERVATION:

The AI successfully identified syntax errors such as missing colons and incorrect indentation, as well as logical errors like variable mismatch. After correction, the function executed properly and returned the expected output. This demonstrates effective automated error detection and correction.

## Task Description -2(Code Style Standardization)

### Task:

Use AI to refactor Python code to comply with standard coding style guidelines.

### Sample Input Code:

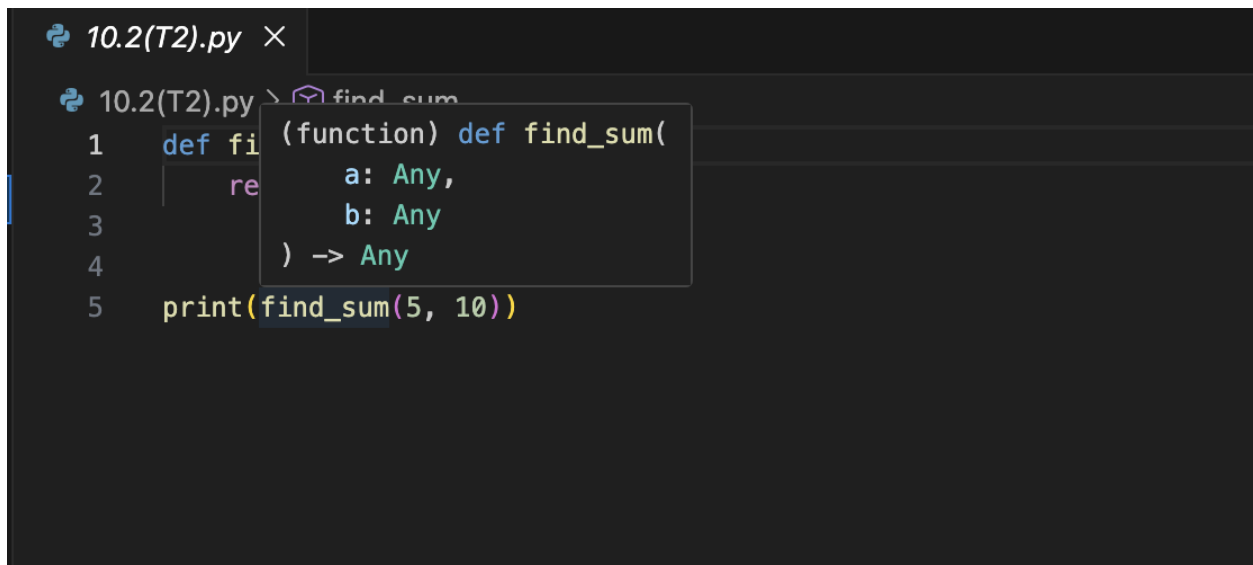
```
def findSum(a,b):return a+b
```

```
print(findSum(5,10))
```

**Expected Output-2:**

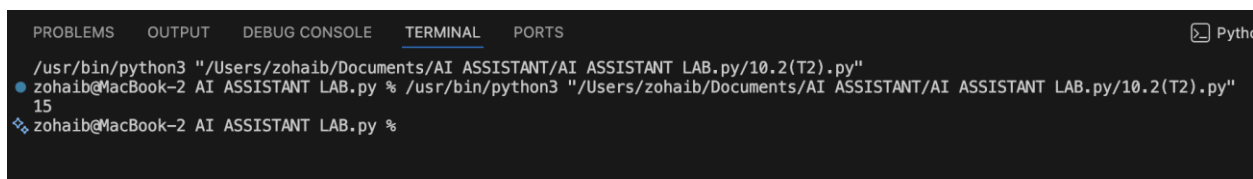
**Well-structured, consistently formatted Python code following standard style conventions.**

**CODE:**



```
10.2(T2).py ×
10.2(T2).py > find_sum
1 def find_sum(a: Any, b: Any) -> Any:
2     return a + b
3
4
5 print(find_sum(5, 10))
```

**OUTPUT:**



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python
/usr/bin/python3 "/Users/zohaib/Documents/AI ASSISTANT LAB.py/10.2(T2).py"
zohaib@MacBook-2 AI ASSISTANT LAB.py % /usr/bin/python3 "/Users/zohaib/Documents/AI ASSISTANT LAB.py/10.2(T2).py"
15
zohaib@MacBook-2 AI ASSISTANT LAB.py %
```

**OBSERVATION:**

The AI refactored the code to follow PEP 8 guidelines by improving naming conventions, spacing, and formatting. The updated code is more readable, structured, and maintainable while preserving original functionality..

Task Description -3(Code Clarity Improvement)

Task:

Use AI to improve code readability without changing its functionality.

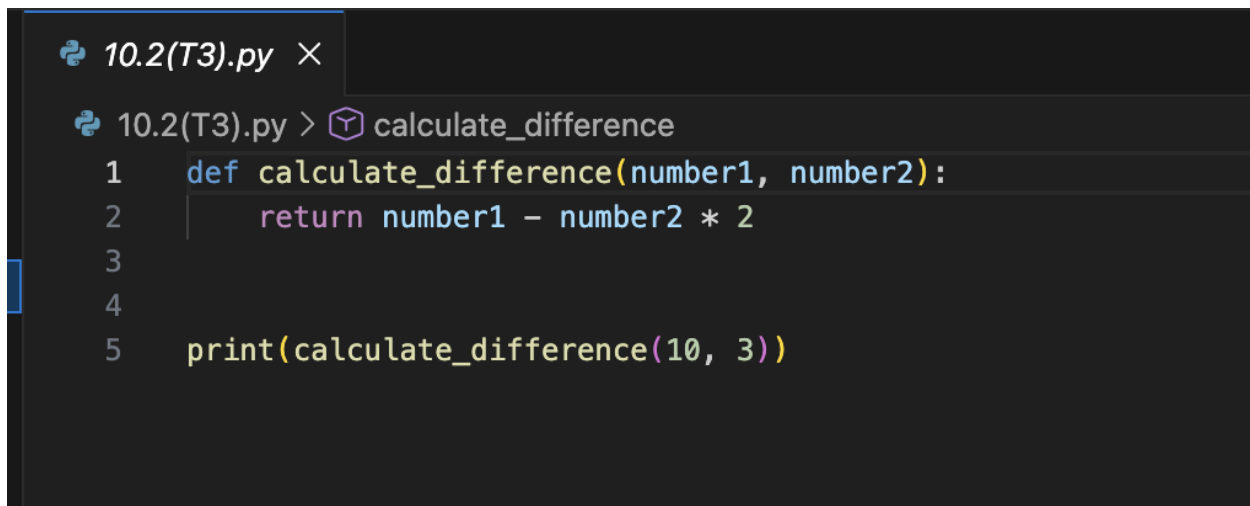
Sample Input Code:

```
def f(x,y):  
    return x-y*2  
print(f(10,3))
```

Expected Output-3:

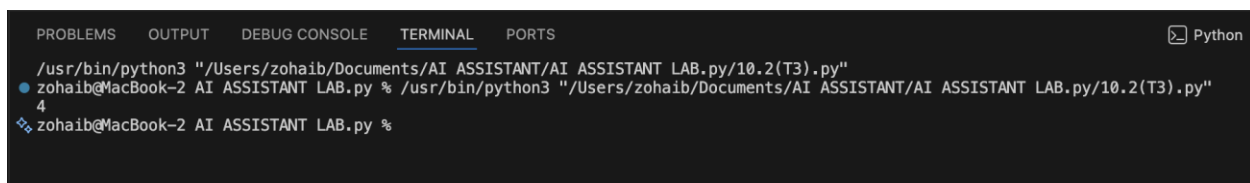
Python code rewritten with meaningful function and variable names, proper indentation, and improved clarity.

CODE:

A screenshot of a code editor with a dark theme. The editor shows a file named '10.2(T3).py'. The code is as follows:

```
1 def calculate_difference(number1, number2):  
2     return number1 - number2 * 2  
3  
4  
5 print(calculate_difference(10, 3))
```

OUTPUT:

A screenshot of a terminal window with a dark theme. The terminal shows the command to run the Python file and the resulting output:

```
/usr/bin/python3 "/Users/zohaib/Documents/AI ASSISTANT/AI ASSISTANT LAB.py/10.2(T3).py"  
zohaib@MacBook-2 AI ASSISTANT LAB.py % /usr/bin/python3 "/Users/zohaib/Documents/AI ASSISTANT/AI ASSISTANT LAB.py/10.2(T3).py"  
4  
zohaib@MacBook-2 AI ASSISTANT LAB.py %
```

OBSERVATION:

The AI improved code readability by replacing unclear function and variable names with meaningful ones. The logic remained unchanged, but the code became easier to understand and maintain.

## Task Description -4(Structural Refactoring)

Task:

Use AI to refactor repetitive code into reusable functions.

Sample Input Code:

```
print("Hello Ram")
```

```
print("Hello Sita")
```

```
print("Hello Ravi")
```

**Expected Output-4:**

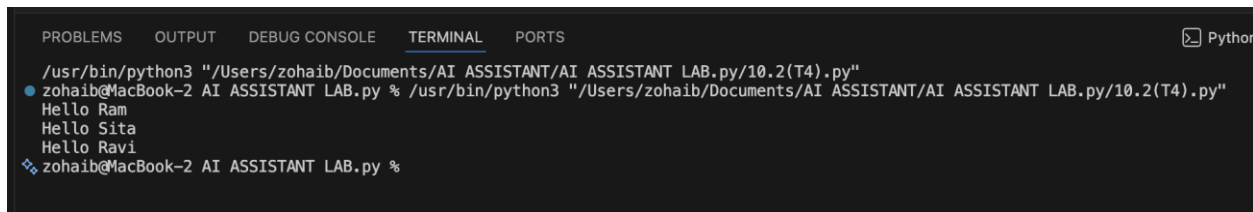
**Modular Python code using reusable functions to eliminate repetition.**

**CODE:**



```
10.2(T4).py ×  
10.2(T4).py > greet  
1 def greet(name):  
2     print("Hello", name)  
3  
4  
5 greet("Ram")  
6 greet("Sita")  
7 greet("Ravi")
```

**OUTPUT:**



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS  
Python  
/usr/bin/python3 "/Users/zohaib/Documents/AI ASSISTANT/AI ASSISTANT LAB.py/10.2(T4).py"  
zohaib@MacBook-2 AI ASSISTANT LAB.py % /usr/bin/python3 "/Users/zohaib/Documents/AI ASSISTANT/AI ASSISTANT LAB.py/10.2(T4).py"  
Hello Ram  
Hello Sita  
Hello Ravi  
zohaib@MacBook-2 AI ASSISTANT LAB.py %
```

**OBSERVATION:**

Repetitive code was refactored into a reusable function, improving modularity and reducing redundancy. The updated structure makes the program more scalable and easier to extend.

## **Task Description -5(Efficiency Enhancement)**

**Task:**

Use AI to optimize Python code for better performance.

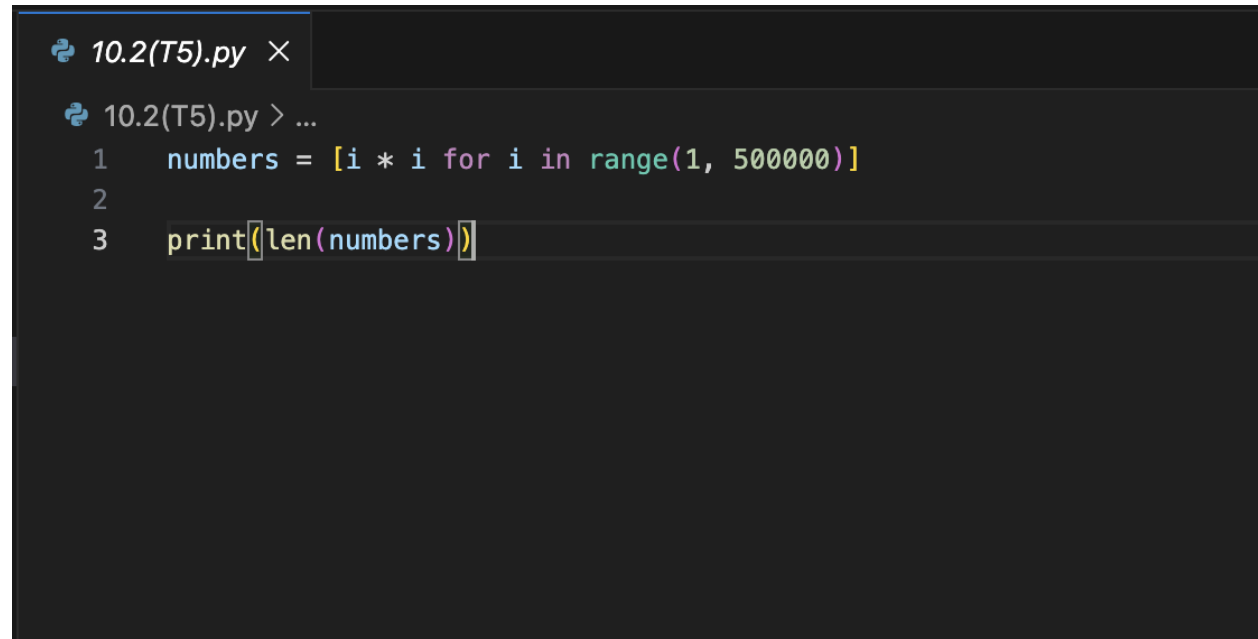
Sample Input Code:

```
numbers = []  
for i in range(1, 500000):  
    numbers.append(i * i)  
print(len(numbers))
```

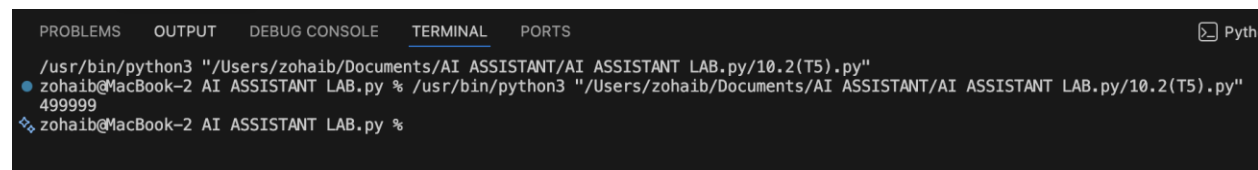
Expected Output-5:

Optimized Python code that achieves the same result with improved performance.

#### CODE:

A screenshot of a code editor window titled '10.2(T5).py'. The editor shows three lines of Python code: 1. 'numbers = [i \* i for i in range(1, 500000)]', 2. (blank line), and 3. 'print(len(numbers))'. The code is written in a dark-themed editor with syntax highlighting.

#### OUTPUT :

A screenshot of a terminal window showing the execution of the Python code. The terminal output is: '/usr/bin/python3 "/Users/zohaib/Documents/AI ASSISTANT/AI ASSISTANT LAB.py/10.2(T5).py"' followed by '499999'. The terminal has tabs for 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL', and 'PORTS'. The 'TERMINAL' tab is active.

#### OBSERVATION :

The AI optimized the program by replacing a loop with list comprehension, improving execution speed and performance. The optimized code produced the same output more efficiently.